(17) Design UNDO & REDO Feature : COMMAND PATTERN

It is Behavioral design pattern that encapsulate object & allowing client to execute some command, without exposing the full methods.

WITHOUT COMMAND PATTERN

AC

```
Plugin;
Stout;
Stop;
turnon(){
  if (Plug)
    sout ("on")
  else sout ("off")
turnoff(){
  if (stop)
    sout ("off")
  else ()
    sout ("Already stop")
```

```
Main.(){
AC = new AC()
AC.plugin()
AC.stout()
AC.turnON()
AC.stop()
AC.turnoff()
```

(1) To turn ON AC, client has to know all steps to TURN ON

(2) But he should just pull Command ON(), then AC should on

(3) Internal steps are not known by client

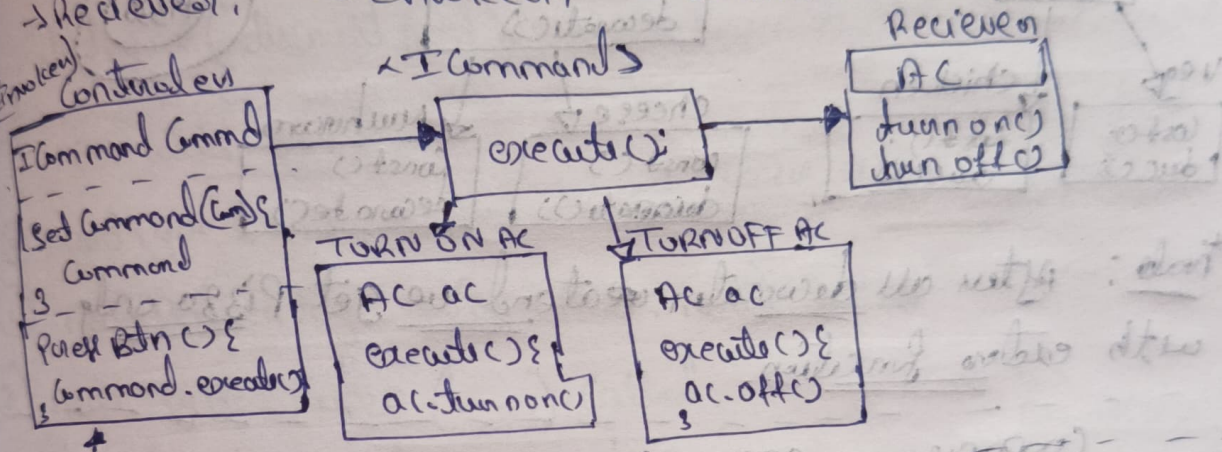
PROBLEMS

① Lack of Abstraction.
Process of turn on AC need more steps & client should aware of all steps.

② UNDO/REDO :- we cannot able to do this feature

③ Difficult for Code maintaining

→ COMMAND PATTERN : Seperate, Logic of
→ Reciever, → Invoker, → Command.



① Client only know to PressBtn.
② Doesn't know internal working
③ If new feature come, to turn on() AC, only AC class will change.

UNDO - REDO

① In Remote Controller, Create 2 stack.
① Store all Commands in 1 stack.
② when undo calls, Pop last Command & add to redo stack.
③ when Redo calls, pop from Redo stack & Call Command Store in normal stack.

① ON
② Temp
③ OFF

